

# SQL and PostgreSQL Fundamentals

## SQL Fundamentals

SQL (Structured Query Language) is used to store, retrieve, and manipulate data in relational databases.

PostgreSQL is a relational database system that implements SQL and is widely used in backend applications.

### What is a Relational Database

A relational database organizes data into tables.

Each table contains:

- rows individual records
- columns attributes of each record

```
users
-----
id | name   | age
-----
1  | Maria  | 30
2  | John   | 25
```

Each row represents one user. Each column describes a property of that user.

### Creating a Table

Tables define how data is structured.

```
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    name TEXT,
    age INTEGER
);
```

Explanation:

- **SERIAL** automatically generates increasing IDs

- PRIMARY KEY uniquely identifies each row
- TEXT, INTEGER data types

## Inserting Data

```
INSERT INTO users (name, age)
VALUES ('Maria', 30);
```

Insert multiple rows:

```
INSERT INTO users (name, age)
VALUES
('Maria', 30),
('John', 25);
```

## Reading Data (SELECT)

Retrieve all data:

```
SELECT * FROM users;
```

Retrieve specific columns:

```
SELECT name, age FROM users;
```

Filter data:

```
SELECT * FROM users
WHERE age > 25;
```

## Updating Data

```
UPDATE users
SET age = 31
WHERE name = 'Ioanna';
```

## Deleting Data

```
DELETE FROM users
WHERE name = 'John';
```

## Conditions and Operators

Conditions are used to filter results.

- = equal
- > greater than
- < less than
- AND, OR

```
SELECT * FROM users
WHERE age > 20 AND name = 'Maria';
```

## Sorting and Limiting Results

```
SELECT * FROM users
ORDER BY age DESC
LIMIT 2;
```

Explanation:

- ORDER BY sorts results
- DESC means descending order
- LIMIT restricts number of rows returned

## Relationships Between Tables

Real applications use multiple related tables.

```
CREATE TABLE orders (
    id SERIAL PRIMARY KEY,
    user_id INTEGER REFERENCES users(id),
    amount INTEGER
);
```

Explanation:

- `user_id` connects orders to users
- `REFERENCES` creates a foreign key relationship

## JOIN (Combining Tables)

```
SELECT users.name, orders.amount  
FROM users  
JOIN orders ON users.id = orders.user_id;
```

JOIN allows combining data from multiple tables into a single result.

## PostgreSQL Features

PostgreSQL extends SQL with additional capabilities:

- strong type system
- JSON support
- indexing for performance

```
CREATE TABLE products (  
    id SERIAL PRIMARY KEY,  
    data JSONB  
);
```

## Typical Workflow in an Application

In a backend system:

1. tables are created to define data structure
2. data is inserted or updated
3. queries retrieve data using `SELECT`
4. results are sent to the frontend

SQL is used throughout the entire lifecycle of data handling.

# Working with PostgreSQL and Spring Boot

In a typical backend application, PostgreSQL is used as the database and Spring Boot connects to it using configuration files and dependencies.

## 1. Creating a Database (psql)

To create and manage a database, we use the PostgreSQL terminal tool: `psql`.

```
psql -U postgres
```

Once inside:

```
CREATE DATABASE mydb;
```

List databases:

```
\l
```

Connect to a database:

```
\c mydb
```

## 2. Basic psql Navigation

```
\dt -- list tables  
\d users -- describe table  
\q -- quit psql
```

## 3. Spring Boot Database Configuration

Spring Boot connects to PostgreSQL using the `application.properties` file.

```
spring.datasource.url=jdbc:postgresql://localhost:5432/mydb
spring.datasource.username=postgres
spring.datasource.password=your_password

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

Explanation:

- `url` location of the database
- `username/password` database credentials
- `ddl-auto=update` automatically creates or updates tables

## 4. Required Dependency (Maven)

Spring Boot needs a PostgreSQL driver.

```
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
</dependency>
```

## 5. Creating Tables with JPA

In Spring Boot, tables are usually created from Java classes.

```
@Entity
public class User {

    @Id
    @GeneratedValue
    private Long id;

    private String name;
    private int age;
}
```

When the application runs, Hibernate creates the table automatically.

## 6. Typical Workflow

1. Start PostgreSQL
2. Create database using `psql`
3. Configure Spring Boot connection

4. Define entities (Java classes)
5. Run application tables are created
6. Query data using SQL or API endpoints